

N3 Reasoning

Simon Frost

Overview

Notation3 (N3) extends RDF with formulas (quoted graphs), variables, and implication rules. RDFLib.jl includes a full N3 reasoner based on the Euler Abstract Machine (EAM) with support for 100+ built-in predicates covering math, string, list, logic, cryptography, and time operations.

```
using RDFLib
```

Basic Forward Chaining

Rules in N3 use => (log:implies) to express “if antecedent then consequent”:

```
n3 = """
@prefix : <urn:example:> .

:Socrates a :Human .
:Plato a :Human .
:Fido a :Dog .

{ ?X a :Human } => { ?X a :Mortal } .
"""

g = parse_n3(n3)
result = reason(g)

println("Mortals:")
for t in triples(result, (nothing, RDF.type, URIRef("urn:example:Mortal")))
    println("  ", t.subject)
end
```

Mortals:

```
URIRef("urn:example:Socrates")
URIRef("urn:example:Plato")
```

Chained Rules

Rules can chain — the conclusion of one rule feeds the antecedent of another:

```
n3 = """
@prefix : <urn:example:> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .

:Cat rdfs:subClassOf :Feline .
:Feline rdfs:subClassOf :Mammal .
:Mammal rdfs:subClassOf :Animal .
:whiskers a :Cat .

{ ?X a ?A . ?A rdfs:subClassOf ?B } => { ?X a ?B } .
"""

g = parse_n3(n3)
result = reason(g)

println("Whiskers is a:")
for t in triples(result, (URIRef("urn:example:whiskers"), RDF.type, nothing))
    println("  ", t.object)
end
```

Whiskers is a:

```
URIRef("urn:example:Animal")
URIRef("urn:example:Feline")
URIRef("urn:example:Mammal")
URIRef("urn:example:Cat")
```

Backward Chaining

Rules using `<=` fire backward (goal-directed):

```

n3 = """
@prefix : <urn:example:> .
@prefix math: <http://www.w3.org/2000/10/swap/math#> .

:alice :age 25 .
:bob :age 17 .
:carol :age 30 .

{ ?X :isAdult true } <= { ?X :age ?A . ?A math:notLessThan 18 } .

:alice :isAdult ?R1 .
:bob :isAdult ?R2 .
"""

g = parse_n3(n3)
result = reason(g)

for t in triples(result, (nothing, URIRef("urn:example:isAdult"), nothing))
    println("${t.subject} isAdult ${t.object}")
end

```

```
URIRef("urn:example:alice") isAdult Variable("R1")
```

```
URIRef("urn:example:bob") isAdult Variable("R2")
```

Math Builtins

N3 provides built-in predicates for arithmetic:

```

n3 = """
@prefix : <urn:example:> .
@prefix math: <http://www.w3.org/2000/10/swap/math#> .

:Let :values (10 3) .

{
    :Let :values (?A ?B) .
    (?A ?B) math:sum ?sum .
    (?A ?B) math:difference ?diff .
    (?A ?B) math:product ?prod .
    (?A ?B) math:quotient ?quot .
    ?A math:absoluteValue ?abs .
}

```

```

}
=>
{
    :result :sum ?sum ;
        :difference ?diff ;
        :product ?prod ;
        :quotient ?quot ;
        :absoluteValue ?abs .
} .
"""

g = parse_n3(n3)
result = reason(g)

ex = Namespace("urn:example:")
for (prop, label) in [("sum", "Sum"), ("difference", "Diff"), ("product", "Prod"),
                    ("quotient", "Quot"), ("absoluteValue", "Abs")]
    for t in triples(result, (ex("result"), ex(prop), nothing))
        println("$label: ", t.object)
    end
end

```

```

Sum: Literal("13", datatype=URIRef("http://www.w3.org/2001/XMLSchema#integer"))
Diff: Literal("7", datatype=URIRef("http://www.w3.org/2001/XMLSchema#integer"))
Prod: Literal("30", datatype=URIRef("http://www.w3.org/2001/XMLSchema#integer"))
Quot: Literal("3.3333333333333335", datatype=URIRef("http://www.w3.org/2001/XMLSchema#decimal"))
Abs: Literal("10", datatype=URIRef("http://www.w3.org/2001/XMLSchema#integer"))

```

String Builtins

```

n3 = """
@prefix : <urn:example:> .
@prefix string: <http://www.w3.org/2000/10/swap/string#> .

:input :text "Hello, World!" .

{
    :input :text ?T .
    ?T string:length ?len .
    ?T string:upperCase ?upper .
}

```

```

    ?T string:lowerCase ?lower .
    ("Hello" "Goodbye") string:concatenation ?replaced .
}
=>
{
    :result :length ?len ;
        :upper ?upper ;
        :lower ?lower ;
        :replaced ?replaced .
} .
"""

g = parse_n3(n3)
result = reason(g)

ex = Namespace("urn:example:")
for prop in ["length", "upper", "lower", "replaced"]
    for t in triples(result, (ex("result"), ex(prop), nothing))
        println("$prop: ", t.object)
    end
end
end

```

```

length: Literal("13", datatype=URIRef("http://www.w3.org/2001/XMLSchema#integer"))
upper: Literal("HELLO, WORLD!")
lower: Literal("hello, world!")
replaced: Literal("HelloGoodbye")

```

List Builtins

```

n3 = """
@prefix : <urn:example:> .
@prefix list: <http://www.w3.org/2000/10/swap/list#> .

:data :items (5 3 1 4 2) .

{
    :data :items ?L .
    ?L list:length ?len .
    ?L list:sort ?sorted .
    ?L list:first ?first .
}

```

```

    ?L list:last ?last .
    (10 20 30) list:append ?appended .
}
=>
{
    :result :length ?len ;
          :sorted ?sorted ;
          :first ?first ;
          :last ?last ;
          :appended ?appended .
} .
"""

g = parse_n3(n3)
result = reason(g)

ex = Namespace("urn:example:")
for prop in ["length", "first", "last"]
    for t in triples(result, (ex("result"), ex(prop), nothing))
        println("$prop: ", t.object)
    end
end

```

```

length: Literal("5", datatype=URIRef("http://www.w3.org/2001/XMLSchema#integer"))
first: Literal("5", datatype=URIRef("http://www.w3.org/2001/XMLSchema#integer"))
last: Literal("2", datatype=URIRef("http://www.w3.org/2001/XMLSchema#integer"))

```

Cryptographic Builtins

The N3 cryptographic builtins (`crypto:md5`, `crypto:sha`) require literals as triple subjects. RDFLib.jl permits this inside N3 formula contexts (`{ ... }`) while still enforcing strict RDF semantics for top-level graphs.

```

n3 = """
@prefix : <urn:example:> .
@prefix crypto: <http://www.w3.org/2000/10/swap/crypto#> .

{
    "hello world" crypto:md5 ?md5 .
    "hello world" crypto:sha ?sha .
}

```

```

=>
{
    :result :md5 ?md5 ;
        :sha ?sha .
} .
"""

g = parse_n3(n3)
result = reason(g)

ex = Namespace("urn:example:")
for prop in ["md5", "sha"]
    for t in triples(result, (ex("result"), ex(prop), nothing))
        println("$prop: ", t.object)
    end
end

```

```

md5: Literal("5eb63bbbe01eeed093cb22bb8f5acdc3")
sha: Literal("2aae6c35c94fcfb415dbe95f408b9ce91ee846ed")

```

Log Builtins

```

# log:equalTo - structural equality
n3 = """
@prefix : <urn:example:> .
@prefix log: <http://www.w3.org/2000/10/swap/log#> .

:a :val (1 2 3) .
:b :val (1 2 3) .

{
    :a :val ?X .
    :b :val ?Y .
    ?X log:equalTo ?Y .
}
=>
{
    :result :listsEqual true .
} .
"""

```

```

g = parse_n3(n3)
result = reason(g)

ex = Namespace("urn:example:")
eq = any(t -> t.subject == ex("result"), result)
println("Lists are equal: $eq")

```

Lists are equal: true

Meta-Reasoning: log:conclusion

The log:conclusion builtin runs the reasoner on a formula — enabling meta-reasoning:

```

n3 = """
@prefix : <urn:example:> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix log: <http://www.w3.org/2000/10/swap/log#> .

:theory rdf:value {
    :Cat rdfs:subClassOf :Animal .
    :whiskers a :Cat .
    { ?X a ?A . ?A rdfs:subClassOf ?B } => { ?X a ?B } .
} .

{
    :theory rdf:value ?T .
    ?T log:conclusion ?C .
    ?C log:includes { :whiskers a :Animal } .
}
=>
{
    :test :proved true .
} .
"""

g = parse_n3(n3)
result = reason(g)

ex = Namespace("urn:example:")

```



```
proved = any(t -> t.subject == ex("test") && t.object == Literal(true), result)
println("Meta-reasoning proved whiskers is an Animal: $proved")
```

Meta-reasoning proved whiskers is an Animal: true

A Practical Example: Family Relationships

```
n3 = ""
@prefix : <urn:family:> .
@prefix log: <http://www.w3.org/2000/10/swap/log#> .

:alice :hasChild :bob .
:alice :hasChild :carol .
:bob :hasChild :dave .
:bob :hasChild :eve .
:carol :hasChild :frank .

:alice :gender "female" .
:bob :gender "male" .
:carol :gender "female" .
:dave :gender "male" .
:eve :gender "female" .
:frank :gender "male" .

# Grandparent rule
{ ?X :hasChild ?Y . ?Y :hasChild ?Z } => { ?X :hasGrandchild ?Z } .

# Sibling rule
{ ?P :hasChild ?X . ?P :hasChild ?Y . ?X log:notEqualTo ?Y }
  => { ?X :sibling ?Y } .

# Uncle/Aunt rule
{ ?P :hasChild ?U . ?P :hasChild ?Parent .
  ?U log:notEqualTo ?Parent .
  ?Parent :hasChild ?Child .
  ?U :gender "male" }
  => { ?U :uncleOf ?Child } .

{ ?P :hasChild ?A . ?P :hasChild ?Parent .
  ?A log:notEqualTo ?Parent .
```

```

?Parent :hasChild ?Child .
?A :gender "female" }
=> { ?A :auntOf ?Child } .
"""

g = parse_n3(n3)
result = reason(g)

fam = Namespace("urn:family:")
println("Grandchildren of Alice:")
for t in triples(result, (fam("alice"), fam("hasGrandchild"), nothing))
    println("  ", t.object)
end

println("\nSiblings:")
seen = Set()
for t in triples(result, (nothing, fam("sibling"), nothing))
    pair = (min(string(t.subject), string(t.object)), max(string(t.subject), string(t.object)))
    if pair ∉ seen
        push!(seen, pair)
        println("  $(t.subject)  $(t.object)")
    end
end

println("\nUncle/Aunt relationships:")
for t in triples(result, (nothing, fam("uncleOf"), nothing))
    println("  $(t.subject) is uncle of $(t.object)")
end
for t in triples(result, (nothing, fam("auntOf"), nothing))
    println("  $(t.subject) is aunt of $(t.object)")
end
end

```

Grandchildren of Alice:

```

  URIRef("urn:family:dave")
  URIRef("urn:family:eve")
  URIRef("urn:family:frank")

```

Siblings:

```

  URIRef("urn:family:dave")  URIRef("urn:family:eve")
  URIRef("urn:family:carol")  URIRef("urn:family:bob")

```

Uncle/Aunt relationships:

URIRef("urn:family:bob") is uncle of URIRef("urn:family:frank")
URIRef("urn:family:carol") is aunt of URIRef("urn:family:dave")
URIRef("urn:family:carol") is aunt of URIRef("urn:family:eve")